

Windows 内核级攻防：从 PEB 遍历到 EDR 规避

实验环境硬性要求：

- VMware Workstation 虚拟机 (Win10 22H2 专业版)
- 仅主机模式 (Host-Only)，严禁桥接或 NAT 联网
- Visual Studio 2022 (含“使用 C++ 的桌面开发”)
- Python 3.x (第 68 课、第 69 课需要编写编码器脚本)
- 可选工具：Process Hacker、WinDbg、CFF Explorer、VirusTotal (VT)

上部：注入与规避

第 1 周：PEB 遍历与动态 API 解析

第 1 课

- ✧ 课程名称：PEB 结构深度解析与内存定位
- ✧ 课程内容：学习 Windows 进程环境块 (PEB) 的结构，理解 `gs:[0x60]` 如何指向 TEB 进而找到 PEB，掌握 `PEB_LDR_DATA` 中 `InMemoryOrderModuleList` 双向链表的遍历方法。通过手工偏移计算获取 `kernel32.dll` 和 `ntdll.dll` 的基址，完全不依赖 `GetModuleHandle` API。
- ✧ 要求掌握内容：能手动编写代码遍历 PEB 链表获取已加载模块基址，理解 `LDR_DATA_TABLE_ENTRY` 结构体中 `DllBase` 和 `BaseDllName` 字段的含义。
- ✧ 相关知识点：TEB/PEB 结构、`gs` 段寄存器、双向链表遍历、`CONTAINING_RECORD` 宏

第 2 课

- ✧ 课程名称：WOW64 兼容性检测与 32/64 位 PEB 差异
- ✧ 课程内容：学习 `NtQueryInformationProcess` API，使用 `ProcessWow64Information` 参数检测目标进程是否为 32 位 (WOW64)。理解 64 位与 32 位进程 PEB 结构偏移差异 (`wow64ext!peb`)，学习 `NtWow64ReadVirtualMemory64` 在跨模式读取时的方法。
- ✧ 要求掌握内容：能在代码中判断目标进程模式，64 位版本为必修实现，WOW64 拓展作为理论了解不强制编码。
- ✧ 相关知识点：`NtQueryInformationProcess`、WOW64 机制、`ProcessWow64Information`

第 3 课

- ✧ 课程名称：PE 文件导出表结构与 ROR13 哈希算法
- ✧ 课程内容：解析 `IMAGE_DOS_HEADER` 和 `IMAGE_NT_HEADERS` 结构，定位 `IMAGE_DATA_DIRECTORY` 中的导出表 (Export Table)。学习 ROR13 哈希算法 (循环右移 13 位)，对导出表中的函数名计算哈希值并与硬编码值比对，从而在不使用明文字符串的情况下定位 `CreateRemoteThread` 等敏感 API。
- ✧ 要求掌握内容：能手写 ROR13 哈希函数，理解导出表遍历逻辑，知道如何从 `AddressOfNames`、`AddressOfFunctions`、`AddressOfNameOrdinals` 三个数组中提取函数地址。
- ✧ 相关知识点：PE 文件格式、`IMAGE_EXPORT_DIRECTORY`、导出表三数组结构

第 4 课

- ✧ 课程名称：封装 GetProcAddressR——纯运行时 API 解析器
- ✧ 课程内容：将前几课的内容整合封装为通用函数 GetProcAddressR(HMODULE hModule, DWORD hash)，该函数接受模块基址和哈希值，返回对应函数的地址。编写测试代码验证动态解析 VirtualAllocEx 和 CreateRemoteThread。
- ✧ 要求掌握内容：能独立写出完整的 GetProcAddressR 函数，并用它替代原生的 GetProcAddress 实现注入器的 API 解析。
- ✧ 相关知识点：函数指针类型转换、模块基址获取、哈希值预计算

第 5 课

- ✧ 课程名称：IAT 隐藏验证——IDA 静态分析检验
- ✧ 课程内容：将编译好的 EXE 用 IDA（或 CFF Explorer）打开，检查导入地址表（IAT）中是否出现 CreateRemoteThread、VirtualAllocEx 等敏感 API 名称。对比原生 GetProcAddress 版本的 IAT 差异，理解静态查杀的绕过原理。
- ✧ 要求掌握内容：能独立完成 IAT 检验流程，确保 EXE 的导入表中仅出现 ExitProcess 等基础函数。
- ✧ 相关知识点：导入地址表（IAT）、PE 导入节、静态查杀原理

第 6 课

- ✧ 课程名称：Detection——PEB 遍历的蓝队检测视角
- ✧ 课程内容：分析蓝队如何检测 gs:[0x60]访问异常，学习 NtQueryInformationProcess 的 ProcessDebugPort 检测调试器附着。讨论反制策略：将遍历逻辑拆分为多个小函数、使用间接调用、或通过异常处理隐藏 PEB 访问特征。
- ✧ 要求掌握内容：理解 PEB 遍历的可检测性，能列举至少两种检测手段和对应的反制策略。
- ✧ 相关知识点：NtQueryInformationProcess、ProcessDebugPort、反调试技术

第 7 课

- ✧ 课程名称：综合练习——无 IAT 版注入器雏形
- ✧ 课程内容：将本周所学全部整合，编写一个完整的控制台程序，通过 PEB 遍历 + 导出表哈希解析，动态获取 VirtualAllocEx、WriteProcessMemory、CreateRemoteThread，向目标进程注入一段简单的 Shellcode（如 mov eax, 42; ret）。完成后用 IDA 验证 IAT 隐藏效果。
- ✧ 要求掌握内容：独立完成无 IAT 版注入器的完整编码与调试，能解释每一步的原理。
- ✧ 相关知识点：注入器完整流程、Shellcode 构造、远程线程创建

第 2 周：模块蹂躏（Module Stomping）

第 8 课

- ✧ 课程名称：从进程镂空到模块蹂躏——为什么放弃 NtUnmapViewOfSection
- ✧ 课程内容：介绍进程镂空（Process Hollowing）的原理与局限性——在现代 Win10 上 NtUnmapViewOfSection 成功率极低，因 ASLR 和 ACG 导致 VirtualAllocEx 频繁返回 ERROR_INVALID_ADDRESS。引出替代方案：模块蹂躏——直接向已加载模块的 .text

节空隙写入 Shellcode。

- ✧ 要求掌握内容：理解进程镂空失败的根本原因（ASLR/ACG/基址占用），知道模块踩踏的核心思路。
- ✧ 相关知识点：ASLR、ACG（动态代码防护）、NtUnmapViewOfSection、ERROR_INVALID_ADDRESS

第 9 课

- ✧ 课程名称：Code Cave 扫描与.text 节空隙利用
- ✧ 课程内容：使用 EnumProcessModules 枚举目标进程已加载的模块，对每个模块的.text 节进行扫描，定位末尾未使用的空隙（通常为 0x00 或 0xCC 填充区），判断是否足够容纳 Shellcode。若空隙 ≥ 256 字节，用 WriteProcessMemory 写入 Shellcode，用 CreateRemoteThread 启动。
- ✧ 要求掌握内容：能独立编写 Code Cave 扫描函数，理解.text 节的内存布局与空隙产生原因（对齐填充）。
- ✧ 相关知识点：EnumProcessModules、GetModuleInformation、PE 节区结构、内存对齐

第 10 课

- ✧ 课程名称：Detection——模块踩踏的蓝队检测视角
- ✧ 课程内容：分析 EDR 如何检测模块踩踏——监控 WriteProcessMemory 写入.text 节的行为（正常程序极少修改自己的代码段）、检查 CreateRemoteThread 起始地址是否落在合法模块范围内。讨论反制：使用 APC 替代 CreateRemoteThread，或在回调函数中执行。
- ✧ 要求掌握内容：理解模块踩踏的检测原理，能对比其与 VirtualAllocEx 分配的优缺点。
- ✧ 相关知识点：内存属性监控、PAGE_EXECUTE_READ、PAGE_EXECUTE_READWRITE

第 3 周：APC 注入与 Early Bird

第 11 课

- ✧ 课程名称：APC 机制与 NtQueueApcThread 注入原理
- ✧ 课程内容：学习 Windows 异步过程调用（APC）机制，理解每个线程拥有 APC 队列，线程进入可提醒等待状态（SleepEx、WaitForSingleObjectEx）时系统检查队列并执行回调。掌握 NtQueueApcThread 的参数含义，理解用户态 APC 如何在不创建新线程的情况下执行代码。
- ✧ 要求掌握内容：能用手写代码调用 NtQueueApcThread 向目标线程插入 APC，理解可提醒状态的含义。
- ✧ 相关知识点：APC 队列、可提醒状态、SleepEx/WaitForSingleObjectEx、NtQueueApcThread

第 12 课

- ✧ 课程名称：Early Bird 时序攻击——在 EDR 钩子落地前执行
- ✧ 课程内容：学习 Early Bird 的核心思路——创建挂起进程（CREATE_SUSPENDED）时，EDR 的用户态钩子尚未完全落地。在此窗口期通过 APC 注入 Shellcode 到主线

程，恢复线程后 Shellcode 在入口点之前执行。对比 CreateRemoteThread、APC、Early Bird 在任务管理器中的表现差异。

- ✧ 要求掌握内容：能用 CREATE_SUSPENDED + NtQueueApcThread 实现 Early Bird 注入，理解时序窗口的产生原因。
- ✧ 相关知识点：CREATE_SUSPENDED、进程创建时序、EDR 钩子加载时序

第 13 课

- ✧ 课程名称：Detection——APC 注入的蓝队检测视角
- ✧ 课程内容：分析 EDR 如何检测跨进程 APC 注入——监控 NtQueueApcThread 调用（正常程序极少跨进程插入 APC）、检查 CREATE_SUSPENDED 标志的进程创建。讨论反制：使用 NtCreateThreadEx 的隐藏标志、或利用 SetThreadContext 替代 APC。
- ✧ 要求掌握内容：理解 APC 注入的可检测性，能列举至少两种检测手段。
- ✧ 相关知识点：NtCreateThreadEx、SetThreadContext、进程创建标志监控

第 4 周：直接系统调用与间接系统调用

第 14 课

- ✧ 课程名称：x64 系统调用约定与 SSN 提取
- ✧ 课程内容：学习 x64 架构下系统调用的完整流程——kernel32!WriteProcessMemory → ntdll!NtWriteVirtualMemory → syscall → 内核。掌握 x64 调用约定（rcx/rdx/r8/r9 存放前 4 个参数，eax 存放 SSN）。从 ntdll.dll 的 NtWriteVirtualMemory 函数机器码中提取 SSN。
- ✧ 要求掌握内容：能手动提取 SSN，理解 syscall 指令的底层触发机制。
- ✧ 相关知识点：x64 调用约定、SSN（系统服务编号）、ntdll.dll 导出表

第 15 课

- ✧ 课程名称：Shadow Space 栈对齐——防止 BSOD 的关键
- ✧ 课程内容：学习 x64 系统调用中 Shadow Space（32 字节影子空间）的作用。在调用 syscall 前必须 sub rsp, 0x20 分配空间，调用后 add rsp, 0x20 恢复，否则高版本 Windows（1903+）在 KiSystemService 返回时因栈不平衡触发蓝屏。提供安全的汇编模板。
- ✧ 要求掌握内容：能独立写出包含 Shadow Space 的 syscall 内联汇编代码，理解 BSOD 的原因。
- ✧ 相关知识点：sub rsp, 0x20、KiSystemService 返回机制、栈平衡

第 16 课

- ✧ 课程名称：裸 Syscall 封装与测试
- ✧ 课程内容：封装 NtWriteVirtualMemorySyscall 函数，使用内联汇编或 __intrin.h 实现直接 syscall。向计算器进程写入 Shellcode，用 ReadProcessMemory 验证写入是否成功。
- ✧ 要求掌握内容：能独立完成裸 Syscall 的封装与测试，理解其与 kernel32 版本的区别。
- ✧ 相关知识点：__intrin.h、__asm 内联汇编、ReadProcessMemory 验证

第 17 课

- ✧ 课程名称：间接系统调用——调用栈伪造
- ✧ 课程内容：裸 syscall 的 Return Address 指向调用者（MyLab.exe），EDR 通过栈回溯可检测。学习两种调用栈伪造方案：RtlCaptureContext + RtlRestoreContext 切换执行流；jmp [rip]方式将 syscall 指令复制到 ntdll 区域内执行。提供可直接复用的模板代码。
- ✧ 要求掌握内容：能理解间接系统调用的核心思路，能复用模板代码实现调用栈伪造。
- ✧ 相关知识点：RtlCaptureContext、RtlRestoreContext、jmp [rip]、栈回溯检测

第 18 课

- ✧ 课程名称：Detection——Syscall 的蓝队检测视角
- ✧ 课程内容：分析蓝队如何检测直接系统调用——栈回溯中 Return Address 不指向 ntdll、syscall 调用频率异常、调用者地址在非 ntdll 区域。学习 EDR 的栈回溯检测原理，讨论间接 Syscall 的局限性（仍可能在 ETW 内核日志中留下痕迹）。
- ✧ 要求掌握内容：理解栈回溯检测的原理，能对比裸 Syscall 与间接 Syscall 的检测差异。
- ✧ 相关知识点：栈回溯（Stack Walking）、ETW 内核日志、ntdll 区域识别

第 5 周：PPID 欺骗（Parent Process ID Spoofing）

第 19 课

- ✧ 课程名称：PROC_THREAD_ATTRIBUTE_LIST 扩展属性机制
- ✧ 课程内容：学习 CreateProcess 的 lpStartupInfo 参数可传入 STARTUPINFOEX 结构体，其中 lpAttributeList 包含扩展属性列表。掌握 InitializeProcThreadAttributeList、UpdateProcThreadAttribute 等 API 的使用，理解 PROC_THREAD_ATTRIBUTE_PARENT_PROCESS 属性的作用。
- ✧ 要求掌握内容：能独立编写属性列表的初始化代码，理解 PROC_THREAD_ATTRIBUTE_LIST 的内存结构。
- ✧ 相关知识点：STARTUPINFOEX、InitializeProcThreadAttributeList、UpdateProcThreadAttribute

第 20 课

- ✧ 课程名称：PPID 伪造完整实现与验证
- ✧ 课程内容：获取 explorer.exe 的 PID，OpenProcess 获取句柄，将其设置为新进程的父进程句柄。用 CreateProcess 创建子进程，在 Process Hacker 中验证父进程显示为 explorer.exe 而非 cmd.exe。注意需启用 SeAssignPrimaryTokenPrivilege。
- ✧ 要求掌握内容：能独立完成 PPID 欺骗的完整实现，理解提权要求及其原因。
- ✧ 相关知识点：SeAssignPrimaryTokenPrivilege、explorer.exePID 获取、Process Hacker 验证

第 21 课

- ✧ 课程名称：Detection——PPID 欺骗的蓝队检测视角
- ✧ 课程内容：分析 PROCESS_CREATION 的 ETW 事件包含 Creator Token 和 Owner

Token，蓝队通过对比两者是否匹配判断 PPID 是否被伪造。讨论 explorer.exe 通常不创建子进程（除双击启动），异常子进程会触发告警。反制思路：同时伪造 Token 或使用 NtCreateUserProcess 低层级参数。

- ✧ 要求掌握内容：理解 Token 对比检测原理，能列举至少两种反制策略。
- ✧ 相关知识点：Creator Token、Owner Token、NtCreateUserProcess、进程树溯源

第 6 周：用户态 ETW 补丁（已失效警告）

第 22 课

- ✧ 课程名称：ETW 原理与 ntdll!EtwEventWrite 定位
- ✧ 课程内容：学习 ETW（Event Tracing for Windows）的用户态出口函数 ntdll!EtwEventWrite，理解其作为 ETW 日志上报的最终出口。通过 PEB 遍历获取 ntdll.dll 基址，解析导出表定位 EtwEventWrite 的函数地址。
- ✧ 要求掌握内容：能独立定位 EtwEventWrite 地址，理解用户态 ETW 的工作原理。
- ✧ 相关知识点：ETW 架构、ntdll!EtwEventWrite、ntdll!EtwpEventWriteFull

第 23 课

- ✧ 课程名称：内存补丁注入与效果验证
- ✧ 课程内容：用 VirtualProtect 将 EtwEventWrite 所在内存页改为 PAGE_EXECUTE_READWRITE，写入 ret 指令（0xC3）使其直接返回。用 Process Monitor 验证该进程的 ETW 事件是否消失。⚠ 此方法仅对关闭内核遥测的旧版 Win10（1809 前）有效，现代 EDR 已改用内核 ETW（Microsoft-Windows-Threat-Intelligence），用户态补丁完全无效。
- ✧ 要求掌握内容：能完成补丁注入实验，并理解其局限性。能说明为什么在现代 EDR 环境下此方法已失效。
- ✧ 相关知识点：VirtualProtect、Inline Hook、内核 ETW、Microsoft-Windows-Threat-Intelligence

第 24 课

- ✧ 课程名称：Detection——内存补丁的蓝队检测视角
- ✧ 课程内容：分析修改 EtwEventWrite 内存页的行为本身就会被 EDR 标记为“可疑内存修改”，进而触发内存扫描（Memory Scan），转储该进程内存进行深度学习分析。讨论此技术的教学价值：理解 ETW 工作原理，而非实用的绕过手段。
- ✧ 要求掌握内容：理解内存修改的检测原理，能评估此技术在实战中的价值（已基本失效）。
- ✧ 相关知识点：内存扫描（Memory Scan）、深度学习检测、VirtualProtect 监控

第 7 周：回调函数执行与分段解密

第 25 课

- ✧ 课程名称：Windows 回调函数执行机制
- ✧ 课程内容：学习 EnumWindows、SetTimer、CreateTimerQueueTimer 等接受函数指针作为回调参数的 API。理解回调在调用者线程上下文中执行，不创建新线程，因此任务管理器“线程”选项卡中看不到任何变化。

✧ 要求掌握内容：能列举至少 3 种可用于执行代码的回调 API，理解其在线程上下文中的执行特点。

✧ 相关知识点：EnumWindows、SetTimer、CreateTimerQueueTimer、线程上下文

第 26 课

✧ 课程名称：XOR 加密 Shellcode 与原地解密执行

✧ 课程内容：将 Shellcode 以 XOR 加密形式存储在二进制文件的数据段中，运行时由回调函数触发解密函数进行原地解密，执行完毕后立即 memset 擦除。实现分段加载——加密→解密→执行→擦除→恢复，避免静态查杀。

✧ 要求掌握内容：能独立编写 XOR 加密 Shellcode 并通过回调触发解密执行，理解分段加载的安全性优势。

✧ 相关知识点：XOR 加密、分段加载、内存擦除、静态查杀绕过

第 27 课

✧ 课程名称：Detection——回调执行的蓝队检测视角

✧ 课程内容：分析 EDR 如何检测回调执行——监控 EnumWindows 等回调函数的调用来源、检测回调地址是否指向可执行内存（非 PAGE_EXECUTE_READ 区域）。讨论反制：使用 CreateTimerQueueTimer（线程池回调，更隐蔽）或 SetWindowsHookEx。

✧ 要求掌握内容：理解回调执行的检测原理，能对比不同回调 API 的隐蔽程度。

✧ 相关知识点：CreateTimerQueueTimer、SetWindowsHookEx、回调地址监控

第 8 周：反射式 DLL 加载器（毕业设计上部）

第 28 课

✧ 课程名称：反射式 DLL 加载原理与基址计算

✧ 课程内容：学习反射式加载的核心思路——不依赖 LoadLibrary，在内存中手动完成 DLL 的加载。通过汇编获取当前指令指针（RIP），按页对齐（0x1000）找到 DLL 基址。理解位置无关代码（PIC）的重要性。

✧ 要求掌握内容：能通过汇编获取当前执行位置，理解基址计算的方法和原理。

✧ 相关知识点：RIP 寄存器、页对齐、位置无关代码（PIC）、__intrin.h

第 29 课

✧ 课程名称：PE 手动映射——复制 Section 与修复重定位

✧ 课程内容：解析 IMAGE_DOS_HEADER → IMAGE_NT_HEADERS，获取各节（Section）的 VirtualAddress 和 SizeOfRawData，用 VirtualAlloc 分配新内存并逐节复制。遍历 .reloc 节修复所有需要重定位的地址，计算基址差值（实际基址 - 默认基址）并加到每个重定位项上。

✧ 要求掌握内容：能独立完成 PE 节复制和重定位修复的编码，理解重定位表的结构。

✧ 相关知识点：IMAGE_BASE_RELOCATION、.reloc 节、重定位修复、VirtualAlloc

第 30 课

✧ 课程名称：修复导入表（IAT）与调用DllMain

✧ 课程内容：遍历 IMAGE_IMPORT_DESCRIPTOR，对每个导入的 DLL 调用 GetProcAddress（或用第 1 周的动态解析函数）获取函数地址，填充到 IAT 中。找到

AddressOfEntryPoint, 构造 DllMain 调用参数 (hinstDLL、dwReason、lpReserved), 手动调用入口点。

- ✧ 要求掌握内容: 能独立完成导入表修复和 DllMain 调用, 理解 DLL 加载的完整流程。
- ✧ 相关知识点: IMAGE_IMPORT_DESCRIPTOR、IAT、DllMain 调用规范

第 31 课

- ✧ 课程名称: 体积优化——/MT 静态链接与禁用 C++ 异常
- ✧ 课程内容: 学习 /MT (静态链接) 与 /MD (动态链接) 的区别, 理解 /MT 导致的体积膨胀问题。强制使用编译选项 /MT /EHa- /GS- /GL- /Oy- /Oi-, 禁止使用 <iostream>、<string>、<vector>, 仅用 Windows Native API (RtlMoveMemory、RtlZeroMemory)。目标: DLL 体积控制在 50KB 以内, .text 节不超过 32KB。
- ✧ 要求掌握内容: 能配置 VS 项目属性实现体积优化, 理解 C++ 运行时库的链接方式对注入成功率的影响。
- ✧ 相关知识点: /MT vs /MD、/EHa-、C++ 异常机制、RtlMoveMemory、RtlZeroMemory

第 32 课

- ✧ 课程名称: 整合前 7 周技术——ppid-spoof 与 etw-patch
- ✧ 课程内容: 将第 1 周的动态 API 解析、第 4 周的 PPID 欺骗、第 6 周的 ETW 补丁整合进反射式加载器。设计命令行参数 --ppid-spoof 和 --etw-patch, 使加载器在启动时自动执行这些规避操作。
- ✧ 要求掌握内容: 能完成多模块整合, 理解命令行参数解析和模块初始化顺序。
- ✧ 相关知识点: 命令行参数解析、模块初始化顺序、GetCommandLine

第 33 课

- ✧ 课程名称: 反射式 DLL 加载器测试与验证
- ✧ 课程内容: 编译最终的反射式 DLL 加载器, 测试 MyLab.exe --ppid-spoof --etw-patch --load MyPay.dll 的完整流程。用 Process Hacker 验证父进程伪装和 ETW 日志阻断效果, 用 IDA 验证 DLL 体积和 IAT 隐藏效果。
- ✧ 要求掌握内容: 能独立完成加载器的编译、测试和验证, 记录每一步的观察结果。
- ✧ 相关知识点: Process Hacker 验证、IDA 分析、dumpbin /headers

第 8.5 周: COM 与 BSTR 基础缓冲周

第 34 课

- ✧ 课程名称: COM 内存模型与 CComPtr 智能指针
- ✧ 课程内容: 学习 COM (组件对象模型) 的基础——IUnknown 接口的 AddRef/Release 引用计数机制, CoInitialize/CoUninitialize 配对规则。使用 CComPtr<T> 智能指针 (#include <atlbase.h>) 自动管理引用计数, 避免手动 Release 遗漏。
- ✧ 要求掌握内容: 能独立写出包含 CoInitialize/CoUninitialize 的 COM 初始化代码, 理解引用计数管理的必要性。
- ✧ 相关知识点: IUnknown、AddRef/Release、CComPtr、CoInitialize/CoUninitialize

第 35 课

- ✧ 课程名称：VARIANT 与 BSTR 类型处理陷阱
- ✧ 课程内容：学习 BSTR 的内存结构（前 4 字节长度前缀），理解必须用 SysAllocString 分配、SysFreeString 释放的原因。学习 VARIANT 容器——使用前 VariantInit、使用后 VariantClear，理解 VT_BSTR 与 VT_EMPTY 的判断逻辑。
- ✧ 要求掌握内容：能独立处理 BSTR 和 VARIANT 的分配与释放，理解内存泄漏的常见来源。
- ✧ 相关知识点：BSTR、SysAllocString、SysFreeString、VARIANT、VariantInit、VariantClear

第 36 课

- ✧ 课程名称：实战演练——COM 调用 WScript.Shell 启动 calc.exe
- ✧ 课程内容：封装 CComInitializer RAI 类（构造初始化 COM，析构清理）。通过 COM 调用 WScript.Shell 的 Run 方法启动 calc.exe，捕获并处理 HRESULT 错误码（使用 SUCCEEDED 宏）。此演练不涉及攻击，仅验证 COM 调用链的可靠性。
- ✧ 要求掌握内容：能独立完成 COM 调用 calc.exe 的完整流程，理解 RAI 封装的作用。
- ✧ 相关知识点：CComInitializer RAI 类、WScript.Shell、IDispatch、Invoke1、HRESULT 错误码

第 8.75 周：Beacon 对象文件（BOF）入门

第 37 课

- ✧ 课程名称：BOF 原理与架构思维
- ✧ 课程内容：学习 Cobalt Strike Beacon 对象文件（BOF）的设计思想——轻量级、位置无关、不创建新线程、在内存中执行的代码片段。理解 BOF 与反射式 DLL 的区别：BOF 是“函数片段”，DLL 是“完整模块”；BOF 的 go 入口点无静态初始化、无 CRT 依赖。
- ✧ 要求掌握内容：理解 BOF 的核心设计思想，能说明 BOF 与反射式 DLL 的差异。
- ✧ 相关知识点：Beacon 对象文件（BOF）、go 入口点、位置无关代码、无 CRT 依赖

第 38 课

- ✧ 课程名称：将 W1 的 GetProcAddressR 封装为 BOF
- ✧ 课程内容：将第 1 周实现的 GetProcAddressR（无 IAT、无全局变量、无静态初始化）移植到一个 BOF 中。编写 go(char* args, int len) 入口点，用 __declspec(allocate) 或 inline 关键字确保代码在 PE 节中。编译命令：cl /c /GS- /Oy- /Os /O1 bof.c。演示用 cna 脚本加载并执行 BOF。
- ✧ 要求掌握内容：能独立将 PEB 遍历代码封装为 BOF，理解 BOF 的编译约束。
- ✧ 相关知识点：__declspec(allocate)、inline、/GS-、/Oy-、cna 脚本

附录章节：Shellcode 加载器混淆（加密器/打包器）

第 A1 课：XOR 加密 + Base64 编码（基础混淆）

- ✧ 课程名称：Shellcode 静态混淆入门——XOR 与 Base64 编码
- ✧ 课程内容：本课的核心思路是：绝不在 EXE 中直接存储原始 Shellcode。我们将通过“编码器-加载器”分离的模式，实现“硬盘上无害，内存中还原”。
 - 编码器（Python 脚本）：读取原始 Shellcode (.bin) 文件 → 使用密钥（如 0xAA）对每个字节进行 XOR 加密 → 将加密数据编码为 Base64 字符串 → 输出为 C 语言头文件 (shellcode.h)。
 - 加载器（C/C++ 程序）：包含 shellcode.h → 运行时先将 Base64 字符串解码回加密二进制 → 用相同密钥 XOR 解密还原原始 Shellcode → 写入内存并执行（复用 VirtualAlloc 和 CreateThread）。
- ✧ 要求掌握内容：能独立编写 Python 编码脚本实现 XOR+Base64 编码流程；能修改现有 C++ 加载器集成解码和解密功能；理解“硬盘上存储 Base64 字符串，内存中还原 Shellcode”的免杀原理。
- ✧ 相关知识点：XOR 对称加密、Base64 编码、静态特征码绕过、VirtualAlloc / CreateThread

第 A2 课（总第 40 课）：资源加载与进阶混淆（实战伪装）

- ✧ 课程名称：进阶混淆——将 Shellcode 伪装成图片资源
- ✧ 课程内容：将 Shellcode 隐藏到图片等正常资源文件中是更高级的伪装技术。本课将教学生把 Shellcode“塞”进一张图片里，并让加载器从图片中提取并执行。
 - 图片隐写（Steganography）：准备一张普通 BMP/PNG 图片，编写 Python 脚本将编码后的 Shellcode 数据追加到图片文件末尾。图片查看器只读取头部数据，图片正常显示，Shellcode 隐藏在尾部。
 - 资源加载器：在 Visual Studio 中将“伪装”好的图片文件作为资源（Resource）嵌入 EXE。在 C++ 代码中使用 FindResource/LoadResource/LoadResourceEx 从 EXE 自身加载该图片资源，定位到图片数据末尾读取附加的 Shellcode 数据，然后进行 Base64 解码、XOR 解密、执行。
 - 进阶技巧（仅了解）：多层混淆（XOR+RC4+Base64）、SGN 专业 Shellcode 编码器变异、加载器分离（远程拉取实现“无文件”落地）。
- ✧ 要求掌握内容：能使用 Python 脚本将编码 Shellcode 附加到图片末尾；能在 VS 中将图片作为资源嵌入项目；能编写 C++ 代码从资源中提取图片并解析出隐藏的 Shellcode 并执行。理解“伪装”——让杀软看到一个正常的、带有有效图标的程序。
- ✧ 相关知识点：图片隐写（Steganography）、Windows 资源（Resource）管理、FindResource / LoadResource / LoadResourceEx、多层编码与混淆

下部：网络与横向扩展

第 9 周：C2 信标与加密通信

第 41 课

- ✧ 课程名称：Winsock 基础——socket/connect/send/recv
- ✧ 课程内容：学习 Winsock API 的完整流程——WSAStartup 初始化、socket 创建套接字、connect 连接远程地址、send/recv 收发数据、closesocket 关闭、WSACleanup 清理。编写一个简单的 TCP 客户端，连接本地监听端口并发送字符串。
- ✧ 要求掌握内容：能独立编写 Winsock TCP 客户端代码，理解每个 API 的作用和调用顺序。
- ✧ 相关知识点：WSAStartup、socket、connect、send/recv、closesocket、WSACleanup

第 42 课

- ✧ 课程名称：XOR 加密与固定心跳实现
- ✧ 课程内容：封装 XOR 加密/解密函数（使用固定密钥）。编写 Beacon 信标模块，每 30 秒向 C2 服务端发送心跳包，内容包含：进程 PID、主机名、已加载模块列表。实现服务端（监听端）的接收与解密功能。
- ✧ 要求掌握内容：能独立实现 XOR 加密的心跳通信，理解固定周期信标的检测风险。
- ✧ 相关知识点：XOR 加密、心跳包设计、Sleep 定时、服务端监听

第 10 周：LSASS 凭证转储与 PPL 绕过

第 43 课

- ✧ 课程名称：PPL 机制与 SeDebugPrivilege 的局限性
- ✧ 课程内容：学习 PPL（受保护进程轻量级）机制，Win10/11 默认开启，lsass.exe 受 ObRegisterCallbacks 保护。理解仅用 SeDebugPrivilege 打开 PROCESS_ALL_ACCESS 会返回 STATUS_ACCESS_DENIED（错误码 0xC0000022）。分析 PPL 在内核层的实现原理。
- ✧ 要求掌握内容：理解 PPL 的保护机制，能解释为什么 SeDebugPrivilege 不足以打开 lsass.exe。
- ✧ 相关知识点：PPL（受保护进程轻量级）、ObRegisterCallbacks、STATUS_ACCESS_DENIED

第 44 课

- ✧ 课程名称：必修路径——修改注册表禁用 PPL
- ✧ 课程内容：修改注册表 HKLM\SYSTEM\CurrentControlSet\Control\Lsa\RunAsPPL 为 0 (DWORD)，重启 VM 后 OpenProcess(PROCESS_ALL_ACCESS)成功。⚠ 此操作需要重启，且在企业环境中通常被 GPO 禁止，仅限实验环境。提供完整的代码实现（含注册表修改和重启提示）。
- ✧ 要求掌握内容：能独立完成注册表修改、VM 重启、OpenProcess 验证的完整流程。
- ✧ 相关知识点：RegOpenKeyEx、RegSetValueEx、RunAsPPL、GPO 策略

第 45 课

- ✧ 课程名称：MiniDumpWriteDump 转储与双特权启用

- ✧ 课程内容：学习 MiniDumpWriteDump (dbghelp.dll) 的用法。启用双特权：SeDebugPrivilege + SeIncreaseQuotaPrivilege（缺一不可）。DumpType 强制指定为 2 (MiniDumpWithFullMemory)。生成 lsass.dmp 后立即删除。
- ✧ 要求掌握内容：能独立实现 LSASS 转储的完整代码，理解双特权的作用。
- ✧ 相关知识点：MiniDumpWriteDump、MiniDumpWithFullMemory、SeIncreaseQuotaPrivilege、GetLastError

第 46 课

- ✧ 课程名称：📖 选修——PPL 绕过之 Kernel Callback（阅读材料）
- ✧ 课程内容：理解不重启绕过 PPL 的原理——使用 NtOpenProcess 配合 PROCESS_QUERY_LIMITED_INFORMATION，配合 NtReadVirtualMemory 直接从 lsass.exe 特定偏移读取内存，手工解析内存结构提取 NTLM 哈希。⚠️ 此方法在 Win10 22H2+ 已失效，仅供理解原理。
- ✧ 要求掌握内容：理解 Kernel Callback 绕过的基本思路，不要求实际运行。
- ✧ 相关知识点：NtOpenProcess、NtReadVirtualMemory、NTLM 哈希在内存中的存储偏移

第 11 周：高级持久化

第 47 课

- ✧ 课程名称：计划任务 COM 接口——ITaskScheduler
- ✧ 课程内容：复用第 8.5 周的 CComInitializer，通过 COM 接口 CLSID_TaskScheduler 创建计划任务。学习 ITaskService::Connect 连接任务服务、ITaskFolder::RegisterTask 注册任务的方法。理解计划任务与 Run 键值相比的隐蔽性优势。
- ✧ 要求掌握内容：能独立通过 COM 接口创建计划任务，理解其隐蔽性原理。
- ✧ 相关知识点：CLSID_TaskScheduler、ITaskService、ITaskFolder、RegisterTask

第 48 课

- ✧ 课程名称：WMI 持久化——Win32_ScheduledJob
- ✧ 课程内容：通过 WMI 的 Win32_ScheduledJob 类创建计划任务。使用 IWbemLocator::ConnectServer 连接 WMI 命名空间、IWbemServices::GetObject 获取类、IWbemClassObject::SpawnInstance 创建实例、IWbemServices::PutInstance 写入。执行后立即删除任务（仅验证原理）。
- ✧ 要求掌握内容：能独立通过 WMI 创建持久化任务，理解其与计划任务 COM 接口的差异。
- ✧ 相关知识点：IWbemLocator、IWbemServices、Win32_ScheduledJob、IWbemClassObject

第 11.5 周：NTLM 认证与 IPC\$验证

第 49 课

- ✧ 课程名称：NTLM Challenge-Response 机制与传参差异
- ✧ 课程内容：理解远程 WMI 连接中 NTLM vs Kerberos 的降级策略（默认走 NTLM）。掌握本地账户与域账户在 ConnectServer 时 User/Domain 参数的传参差异——本地账户

需传入主机名作为 Domain。理解 IPC\$共享在远程认证中的作用。

- ✧ 要求掌握内容：能正确构造不同账户类型的 ConnectServer 参数，理解 NTLM 认证的基本流程。
- ✧ 相关知识点：NTLM Challenge-Response、Kerberos 降级、IPC\$共享、ConnectServer 参数

第 50 课

- ✧ 课程名称：NetUseAdd 手工建立 IPC\$会话验证凭证
- ✧ 课程内容：编写独立函数，纯手工调用 WNetUseConnection 建立 IPC\$认证会话，验证远程凭证有效性。只有 IPC\$连接成功后，才继续执行后续的 WMI ExecMethod。连接失败则直接报错退出。
- ✧ 要求掌握内容：能独立实现 IPC\$验证函数，理解其作为 WMI 前置步骤的必要性。
- ✧ 相关知识点：WNetUseConnection、WNetCancelConnection2、NETRESOURCE、IPC\$连接

第 12 周：远程执行三件套（WMI + SCM + SchTasks）

第 51 课

- ✧ 课程名称：WMI 远程执行——ExecMethod 调用 Win32_Process.Create
- ✧ 课程内容：在 IPC\$验证通过后，使用 IWbemServices::ExecMethod 调用远程主机的 Win32_Process.Create 方法，启动 notepad.exe（仅演示）。理解 WMI 远程执行的检测特征——产生 4624 登录事件和 5156 连接事件。
- ✧ 要求掌握内容：能独立实现 WMI 远程执行，理解其检测特征。
- ✧ 相关知识点：ExecMethod、Win32_Process.Create、4624/5156 事件

第 52 课

- ✧ 课程名称：SCM 远程服务创建与启动
- ✧ 课程内容：使用 OpenSCManager(remoteMachine, NULL, SC_MANAGER_CREATE_SERVICE)打开远程服务控制管理器，CreateService 创建服务指向指定可执行文件，StartService 启动，立即 DeleteService 删除（不留痕迹）。理解 SCM 的检测特征——4697 服务创建事件。
- ✧ 要求掌握内容：能独立实现 SCM 远程服务创建与清理，理解其与 WMI 的检测差异。
- ✧ 相关知识点：OpenSCManager、CreateService、StartService、DeleteService、4697 事件

第 53 课

- ✧ 课程名称：SchTasks 远程计划任务
- ✧ 课程内容：使用 ITaskScheduler COM 接口远程连接目标主机，创建一次性计划任务（触发后自动删除）。理解 SchTasks 与 WMI/SCM 的静默程度差异——仅产生 5140/5142 文件共享事件（若涉及文件共享），不产生 4624 登录事件。优势：企业环境中最不易被发现的横向手法。
- ✧ 要求掌握内容：能独立实现 SchTasks 远程计划任务创建，理解其静默优势。
- ✧ 相关知识点：ITaskScheduler 远程连接、5140/5142 事件、一次性计划任务

第 12.5 周：RegConnectRegistry 远程注册表

第 54 课

- ✧ 课程名称：RegConnectRegistry 远程连接注册表
- ✧ 课程内容：学习 RegConnectRegistry API，远程连接目标主机的注册表 (RegConnectRegistry(remoteMachine, HKEY_LOCAL_MACHINE, &hRemoteKey))。理解远程注册表的访问权限要求——需要目标主机开启远程注册表服务 (RemoteRegistry，默认禁用)。
- ✧ 要求掌握内容：能独立实现 RegConnectRegistry 远程连接，理解其服务依赖。
- ✧ 相关知识点：RegConnectRegistry、RemoteRegistry 服务、HKEY_LOCAL_MACHINE

第 55 课

- ✧ 课程名称：远程修改注册表实现静默持久化
- ✧ 课程内容：通过远程注册表句柄创建启动项 (Run 键值) 或修改防火墙规则。对比其与 WMI/SCM 的检测差异——不产生 4624 登录事件，仅产生 5140/5142 文件共享事件。理解其适用场景与局限性。
- ✧ 要求掌握内容：能独立实现远程注册表修改，理解其静默优势和使用条件。
- ✧ 相关知识点：RegSetValueEx 远程写入、5140/5142 事件、Run 键值

第 13.5 周：Windows 事件日志操纵

第 56 课

- ✧ 课程名称：wevtutil 命令行清除事件日志
- ✧ 课程内容：学习 wevtutil cl Security 和 wevtutil cl System 命令行清除日志的方法。理解全量清除会产生 1102 事件 (“安全审计日志已清除”)，蓝队会优先关注此事件。讨论全量清除的检测风险。
- ✧ 要求掌握内容：能使用 wevtutil 清除日志，理解 1102 事件的检测含义。
- ✧ 相关知识点：wevtutil、1102 事件、Security 日志、System 日志

第 57 课

- ✧ 课程名称：编程清除与选择性删除最佳实践
- ✧ 课程内容：使用 EvtClearLog API 编程清除日志。学习修改注册表 HKLM\SYSTEM\CurrentControlSet\Control\WMI\Autologger 关闭特定通道审计策略 (需提权)。强调最佳实践：选择性删除 (删除特定时间段的特定事件 ID)，而非全量清除，以减少检测风险。
- ✧ 要求掌握内容：能独立实现 EvtClearLog 编程清除，理解选择性删除与全量清除的检测差异。
- ✧ 相关知识点：EvtClearLog、Autologger 注册表、审计策略、事件 ID 过滤

第 14 周：流量伪装与心跳 Jitter

第 58 课

- ✧ 课程名称：WinHTTP API 与 SNI 伪造
- ✧ 课程内容：学习 WinHTTP API 与 WinINET 的区别——WinINET 受 IE 代理设置影响，不适合稳定 C2。强制要求使用 WinHttpOpen 并显式设置

WINHTTP_ACCESS_TYPE_NO_PROXY。在 WinHttpRequest 中传入伪造的 Host: www.microsoft.com 头，实现 SNI 伪造，使 TLS 握手显示为伪造域名。

- ✧ 要求掌握内容：能独立实现 WinHTTP HTTPS 请求，理解 SNI 伪造的原理。
- ✧ 相关知识点：WinHttpRequest、WINHTTP_ACCESS_TYPE_NO_PROXY、SNI 伪造、TLS 握手

第 59 课

- ✧ 课程名称：LZNT1 压缩降熵与 XOR 加密组合
- ✧ 课程内容：学习 RtlCompressBuffer 使用 LZNT1 压缩算法。先压缩 Shellcode/心跳数据（降低熵值），再 XOR 加密（提供保密性），组合使用避免流量特征过于随机。理解高熵值流量在 Zeek/Darktrace 等流量分析系统上的检测风险。
- ✧ 要求掌握内容：能独立实现 LZNT1 压缩 + XOR 加密的组合流程。
- ✧ 相关知识点：RtlCompressBuffer、LZNT1 压缩、熵值检测、流量分析（Zeek/Darktrace）

第 60 课

- ✧ 课程名称：JSON 协议伪装与心跳抖动（Jitter）
- ✧ 课程内容：将数据伪装成正常 JSON 格式
`{"hostname": "%s", "data": "%s", "timestamp": %d}`，模拟/api/statistics 等正常业务 API。心跳间隔加入抖动（Jitter）：30 秒±5 秒随机偏移，破坏周期性规律，绕过基于时间规律的防火墙检测。
- ✧ 要求掌握内容：能独立实现 JSON 封装和 Jitter 逻辑，理解协议伪装和周期检测的防御原理。
- ✧ 相关知识点：JSON 封装、心跳 Jitter、周期性检测防御

第 15 周：模块化 C2 框架

第 61 课

- ✧ 课程名称：指令分发器架构设计
- ✧ 课程内容：设计 Command 结构体（commandId + payload + parameters），C2 服务端下发 JSON 格式指令，客户端解析并动态加载对应功能模块。理解模块化设计的优势——各功能解耦、独立编译、按需加载。
- ✧ 要求掌握内容：能独立设计指令结构体和 JSON 解析逻辑，理解模块化架构的优缺点。
- ✧ 相关知识点：JSON 解析、指令结构体设计、模块化架构

第 62 课

- ✧ 课程名称：反射式 DLL 编译规范（/MT + /EHa- + PIC）
- ✧ 课程内容：所有横向移动插件（WMI、SCM、LSASS）必须编译为位置无关（PIC）的反射式 DLL。强制编译选项：/MT（静态链接，防止目标进程缺 VC 运行时）、/EHa-（禁用 C++ 异常）、/GS-（禁用安全检查）。使用 dumpbin /headers 检查各节大小，确保.text 节不超过 32KB。
- ✧ 要求掌握内容：能配置 VS 项目属性满足反射式 DLL 编译规范，理解每个编译选项的

必要性。

- ✧ 相关知识点：/MT vs /MD、/EHa-、/GS-、PIC、dumpbin /headers

第 63 课

- ✧ 课程名称：双模式分发器——反射模式与回退模式
- ✧ 课程内容：实现双模式分发器——反射模式：将接收到的插件字节流直接传给 W8 的 ReflectiveLoader 入口点（不落地）；回退模式（仅供调试）：仅当命令行带 --disk-debug 标志时，才调用 LoadLibraryA 从磁盘加载。理解反射模式在隐蔽性上的优势。
- ✧ 要求掌握内容：能独立实现双模式分发逻辑，理解两种模式的适用场景。
- ✧ 相关知识点：ReflectiveLoader 接口、LoadLibraryA、--disk-debug 标志

第 16 周：毕业设计（全链路框架）

第 64 课

- ✧ 课程名称：全链路框架设计——从初始突破到内网漫游
- ✧ 课程内容：串联上部（注入规避）与下部（网络横向），设计完整的框架架构——命令行输入 MyLab.exe --remote 192.168.x.x --username Admin --exec Beacon。规划 6 大模块：初始突破（W1-W8）、持久化（W11）、凭证收集（W10）、横向移动（W12-W12.5）、痕迹清理（W13.5）、C2 通信（W9/W14）。
- ✧ 要求掌握内容：能独立设计全链路框架的模块划分和调用关系，理解各模块的依赖顺序。
- ✧ 相关知识点：模块依赖管理、命令行参数解析、框架初始化流程

第 65 课

- ✧ 课程名称：各模块整合与联调
- ✧ 课程内容：将所有独立模块代码合并到统一的框架中，实现以下流程——PPID 欺骗启动隐蔽进程 → ETW 补丁消除用户态日志 → 计划任务建立持久化 → 转储 LSASS 提取凭证（生成后立即删除）→ 用凭证通过 WMI/SCM/RegConnectRegistry 横向移动 → 加密心跳上报结果。编写统一的错误处理机制。
- ✧ 要求掌握内容：能独立完成所有模块的代码整合，理解模块间的数据传递方式（凭证流转、PID 传递等）。
- ✧ 相关知识点：模块整合、错误处理链、数据传递、GetLastError 传播

第 66 课

- ✧ 课程名称：毕业设计验收与环境搭建
- ✧ 课程内容：搭建至少 3 台 VM（跳板机、域控、工作机）组成的域环境。运行完整框架，验证从初始突破到横向移动的完整链路。记录所有操作日志，编写架构设计文档。完成毕业设计报告。
- ✧ 要求掌握内容：能独立搭建实验环境并完成全链路测试，理解域环境的认证机制。
- ✧ 相关知识点：域环境搭建、Active Directory、日志记录、架构文档撰写

第 16.5 周：终极彩蛋——Sleep Mask 与纯净源 Syscall

第 67 课

- ✧ 课程名称：Sleep Mask——消除 RWX 内存特征

- ✧ 课程内容：Shellcode 执行后，RWX 内存页（可读可写可执行）在内存扫描中极其扎眼。学习 Sleep Mask 机制——Shellcode 执行完毕后调用 VirtualProtect 将内存页改为 PAGE_READONLY（或 PAGE_EXECUTE_READ），需要再次执行时再改回。进阶：使用 NtContinue + CONTEXT 结构体实现“休眠中执行”，在 EDR 扫描窗口（通常 30 秒）内隐藏自身。
- ✧ 要求掌握内容：能独立实现 Sleep Mask 的 VirtualProtect 方案，理解 NtContinue 方案的原理（不强制编码）。
- ✧ 相关知识点：VirtualProtect、PAGE_READONLY、NtContinue、CONTEXT 结构体、内存扫描（Memory Scan）

第 68 课

- ✧ 课程名称：纯净源 Syscall——将 stub 复制到 ntdll 空隙
- ✧ 课程内容：W4 教的是将 syscall stub 复制到 .data 节，但 .data 节被执行标记时仍会被内存扫描检测。学习纯净源方案——遍历 ntdll 的 PE 节区，找到未使用的内存空隙（如 .mrdata 节末尾的填充区），将 syscall stub 复制到该空隙中执行。栈回溯显示“调用来自 ntdll 的 .mrdata”，EDR 更难以区分。
- ✧ 要求掌握内容：理解纯净源 Syscall 的原理，能复用模板代码实现（不强制独立编写，因为需要对 PEB 和节区有极深理解）。
- ✧ 相关知识点：.mrdata 节、ntdll 未使用空隙、PE 节区遍历、栈回溯规避

第 69 课

- ✧ 课程名称：终极整合——将 Sleep Mask 与纯净源 Syscall 整合进毕业设计
- ✧ 课程内容：将 Sleep Mask 和纯净源 Syscall 整合进 W16 的毕业设计框架中，最终命令升级为 MyLab.exe --ppid-spoof --etw-patch --sleep-mask --syscall-hole --load MyPay.dll。验证所有功能协同工作，完成最终的架构设计文档更新。
- ✧ 要求掌握内容：能完成最终整合，理解各模块间的兼容性要求。
- ✧ 相关知识点：模块兼容性、初始化顺序、最终验证